CURZAS
UNCo



Estructuras de Repetición: Del FOR al WHILE

Fundamentos de lógica algorítmica para el control de flujo y ciclos de cantidad desconocida.

Gemini Notebook

El punto de partida: La Repetición Fija (FOR)

Permite repetir un bloque de código una cantidad fija de veces. Cuando la repetición termina, la ejecución del resto del programa continúa normalmente. Su estructura base utiliza una secuencia definida.

```
for <variable> in <secuencia de valores>:
    <cuerpo instrucciones>

# Ejemplo: Contar vocales en una frase
frase = str(input('Frase: '))
cant = 0
for letra in frase.lower():
    if letra in 'aeiou':
        cant += 1
print('Cantidad de vocales: ', cant)
```

La función range(inicio, fin, paso) o una lista manual [1,3,5,7,9] dictan exactamente cuántas veces girará el bucle.

Gemini Notebook

El cambio de paradigma: ¿Qué pasa cuando la cantidad es desconocida?

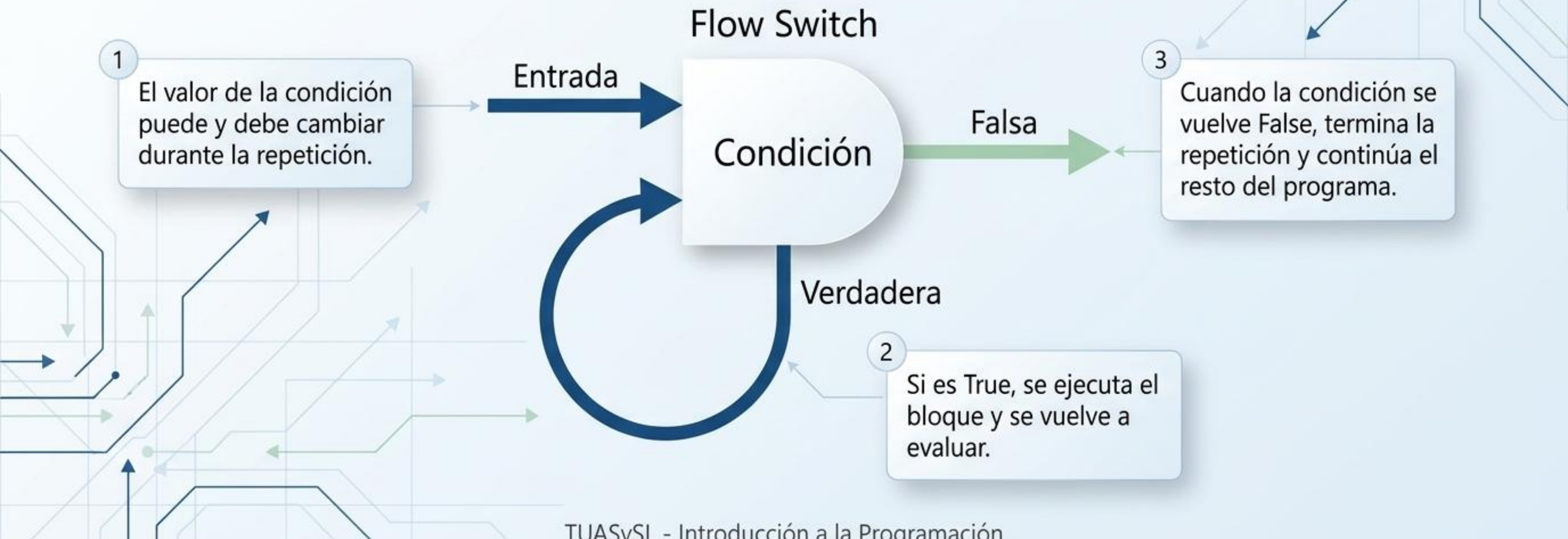
Dimensión	 FOR	 WHILE
Tipo de Repetición	Fija	Condicional
Cuándo usarlo	Cuando conocemos de antemano la cantidad exacta de iteraciones.	Cuando la cantidad de iteraciones depende de eventos externos o cálculos dinámicos.
Mecanismo de Control	Secuencia o rango definido numéricamente.	Evaluación de un valor booleano (Verdadero/Falso).

El ciclo **WHILE** transfiere el control de una simple cuenta matemática a una condición lógica.

Gemini Notebook

Repetición Condicional: La estructura del WHILE

El ciclo while en Python repite un bloque de código MIENTRAS una condición sea verdadera. La ejecución comienza siempre evaluando dicha condición.



Gemini Notebook

El ciclo de vida del WHILE (y el riesgo del bucle infinito)

¡Peligro de Bucle Infinito!

Es crucial garantizar que, en algún momento, la condición se evalúe como **False**. Si el bloque de código no modifica la variable evaluada, el programa se quedará atascado para siempre.



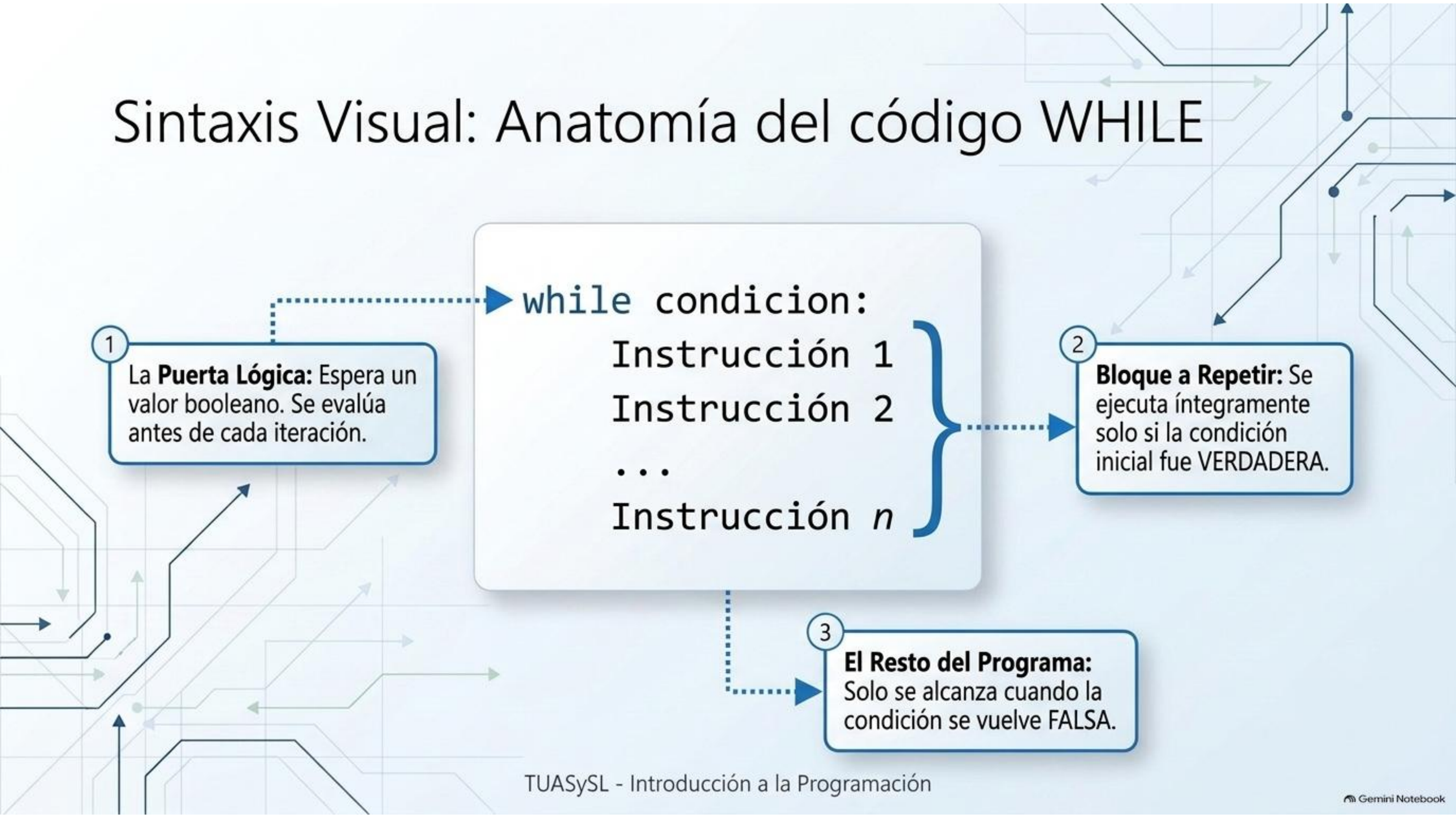
1. Evaluación: Python comprueba si la condición es True.

2. Ejecución: Se corre el bloque de instrucciones interno.

3. Modificación: Se altera la variable de control.

Gemini Notebook

Sintaxis Visual: Anatomía del código WHILE



Gemini Notebook

Caso Práctico 1: Suma Acumulativa de Valores

Necesitamos sumar múltiples números ingresados por el usuario hasta que se ingrese un valor de corte.

```
nro = int(input('Ingrese número: '))
while nro != 0:
    total = total + nro
    nro = int(input('Ingrese número: '))
```

`Total = total + nro`

(El acumulador)

¿Por qué se debe repetir la instrucción input al final del bloque?

Para evitar el bucle infinito. Repetir la lectura del input permite que la variable de la condición cambie de valor en algún momento, logrando que la condición eventualmente sea falsa.

Gemini Notebook

Caso Práctico 2: Validación de Accesos (Anidación)

Dentro de la estructura condicional WHILE podemos armar bloques con otras estructuras, como condicionales if.

```
contrasenia_correcta = 'python2025'
contrasenia_ingresada = ''
while contrasenia_ingresada != contrasenia_correcta:
    contrasenia_ingresada = input('Ingresa la contraseña: ')
    if contrasenia_ingresada != contrasenia_correcta:
        print('Contraseña incorrecta. Pruebe de nuevo.')
print('¡Acceso concedido!')
```

El bucle encierra al usuario hasta que la validación lógica (!=) se convierta en falsa (es decir, cuando acierte la contraseña).

Gemini Notebook

Caso Práctico 3: **Búsqueda y Filtrado de Datos**

El Mayor

Objetivo: Mostrar el mayor número ingresado en una secuencia indefinida.

Lógica: Usamos el WHILE para mantener el programa pidiendo números, y un "if" interno para comparar y guardar solo si el número actual es mayor al anterior guardado.

```python
# Inicialización de lectura y variable del número mayor
numero = int(input("Ingrese un número (0 para salir): "))

#None representa la ausencia de valor. Indica
#que al comenzar la ejecución todavía no hemos procesado ningún número.
mayor = None

# Bucle con condición numérica relacional (numero != 0)
while numero != 0:
    # Guardar si es el primer número o si supera al actual mayor
    if mayor is None or numero > mayor:
        mayor = numero

    # Solicitud del siguiente número para la condición del while
    numero = int(input("Ingrese otro número (0 para salir): "))

print(f"El mayor número ingresado es: {mayor}")

```

Desafío de Adaptación

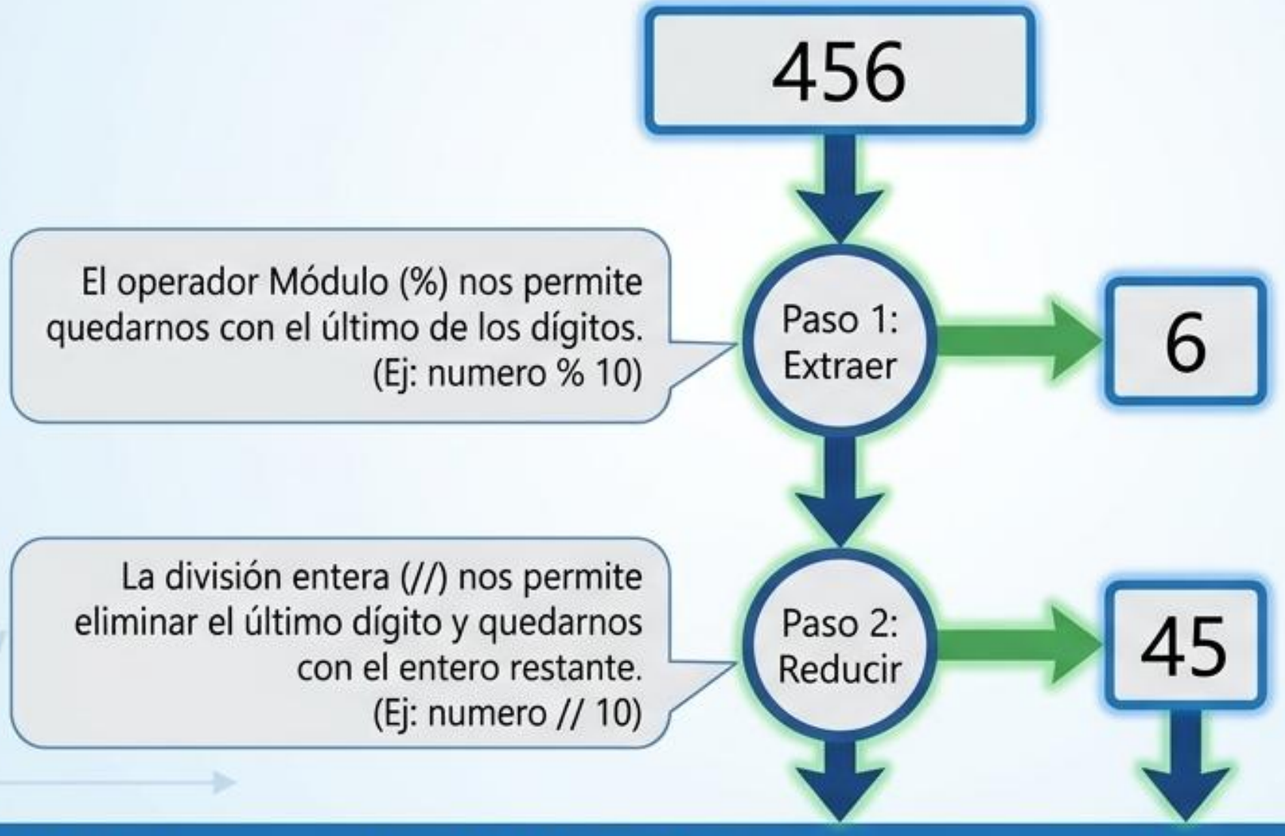
Pregunta: ¿Cómo adaptamos el programa para calcular el **total** de **números positivos** ingresados?

→ **Pista Lógica:** En lugar de comparar con una variable máxima, la condición interna del "if" evaluaría **numero > 0** y sumaríamos a un contador (**cant += 1**).

Gemini Notebook

Caso Práctico 4: Descomposición Matemática

Desarrollamos un algoritmo que nos permita descomponer un número dígito por dígito.
¿Por qué usar While? Porque no sabemos cuántos dígitos tendrá el número que ingrese el usuario.



El ciclo se repite MIENTRAS el número resultante sea mayor a 0.

Gemini Notebook

Síntesis: Las 3 Reglas de Oro del Ciclo WHILE

1. Inicializar

Debes definir un estado o valor inicial antes de que el bucle comience a preguntar. (Ej: contrasenia_ingresada = '')

```
contrasenia_ingresada = ''
```

2. Evaluar

La condición del bucle debe ser una pregunta lógica clara que resulte en Verdadero (para seguir) o Falso (para salir).

```
while contrasenia_ingresada != 'secreto':
```

3. Actualizar (¡Vital!)

Dentro del bloque de código, siempre debe haber una instrucción que modifique la variable evaluada (como volver a pedir un input). Sin esto, creas un bucle infinito.

```
    contrasenia_ingresada = input('Ingrese contraseña: ')
```


Gemini Notebook